Comparative Study of Parallelization Techniques for Word Counting Map Reduce Algorithm

Group 30

314553008 陳姿蓉
314552021 林孝欣
314554046 朱冠熙

Introduction

- Industrial frameworks like Hadoop and Spark abstract parallelization details, but understanding underlying hardware and programming models is crucial for performance tuning.

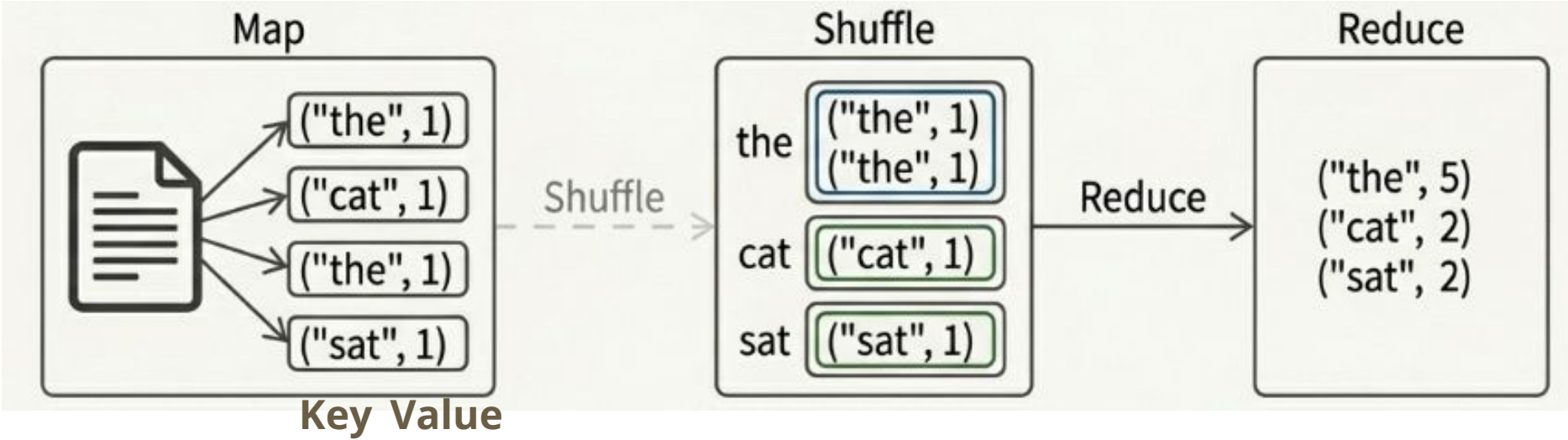
- Evaluate how different parallel programming models perform on the same task — **Word Counting using the MapReduce workflow**.

- Provides insight into when and why each parallel model is effective.

Problem statement

- Implement Word Count using **MapReduce** principles across four distinct parallelization architectures to enable direct quantitative comparison of performance, scalability, and resource utilization.

Problem statement - MapReduce Introduction



Proposed solution

Pthreads

Fine-grained thread control on CPU

- Manual thread creation & Barrier synchronization
- Segment-based data division
- Local intermediate KV-pairs
- Partition-based Merging (Lock-free Reduce)

OpenMP

Loop-level parallelization on CPU

- Pragma-based parallelization
- Simplified implementation
- Automatic workload distribution
- Built-in reduction support

MPI

Distributed computation across nodes

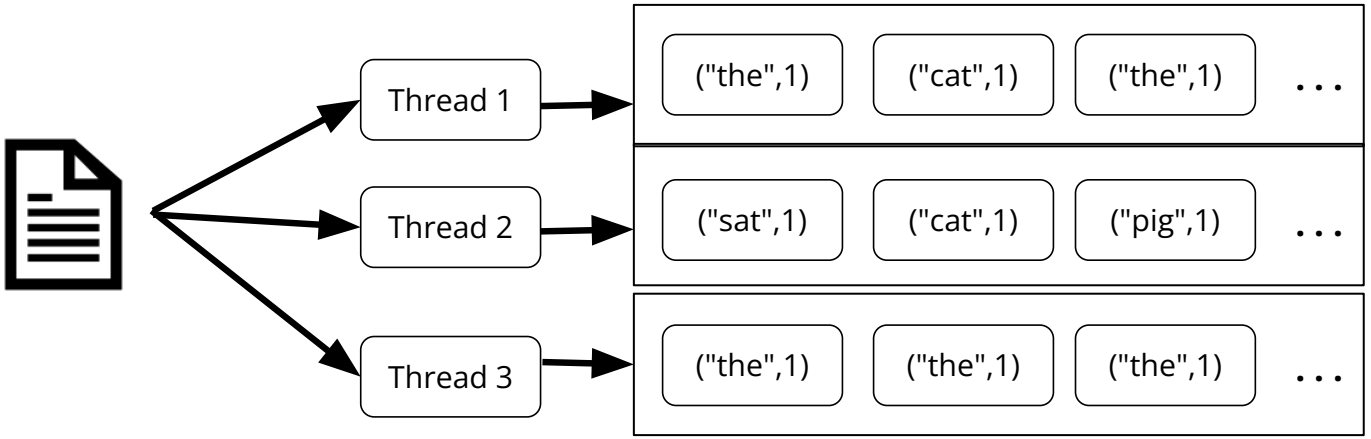
- Multi-process cluster computing
- Message passing communication
- MPI_Reduce for aggregation
- Distributed memory model

CUDA

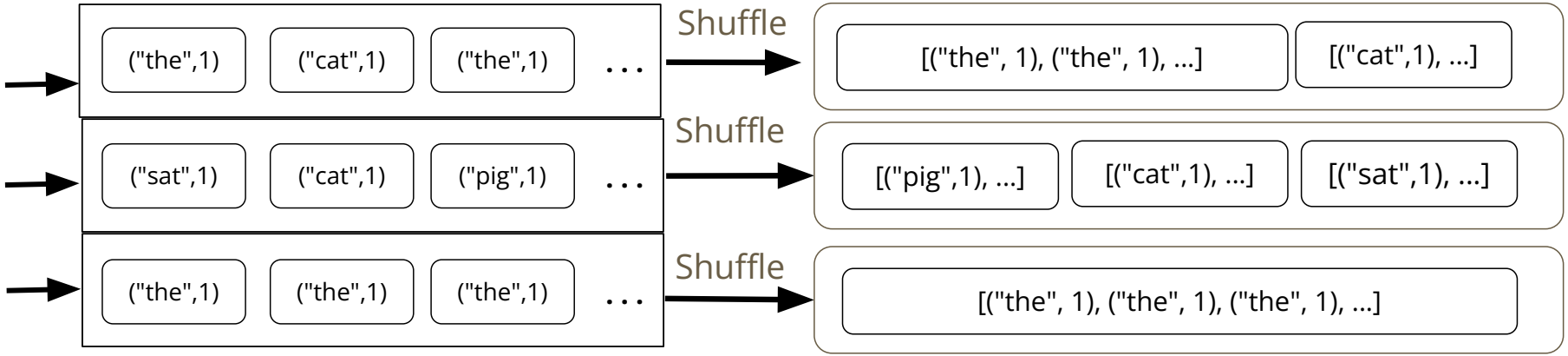
Massive parallelism on GPU

- Block-based GPU thread assignment
- Special word boundary handling

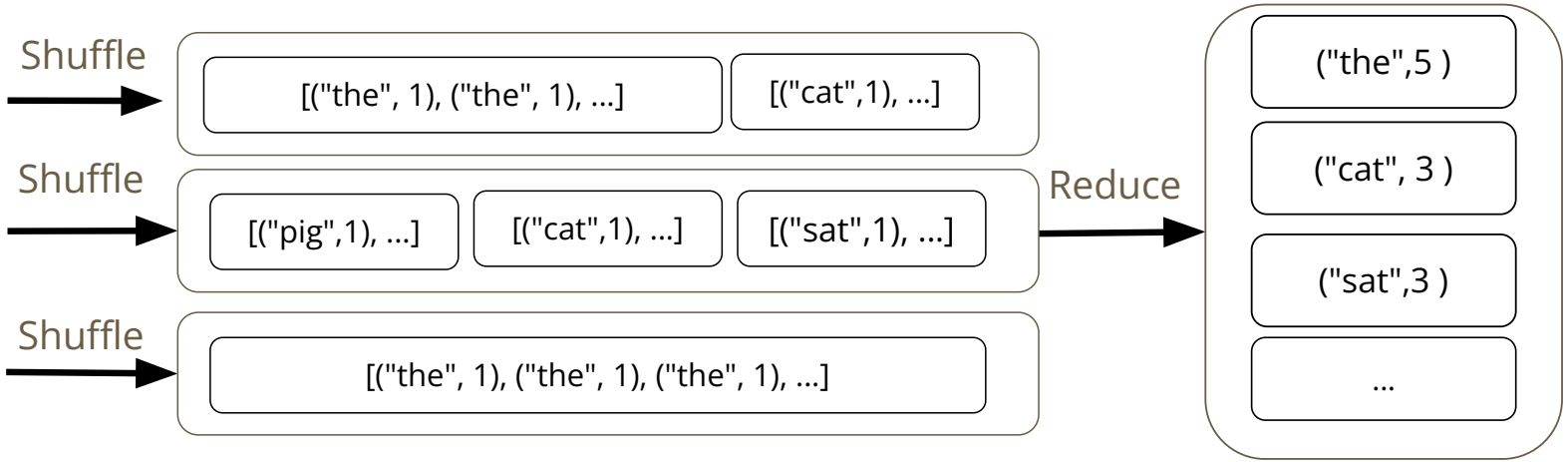
Proposed solution- [Map]



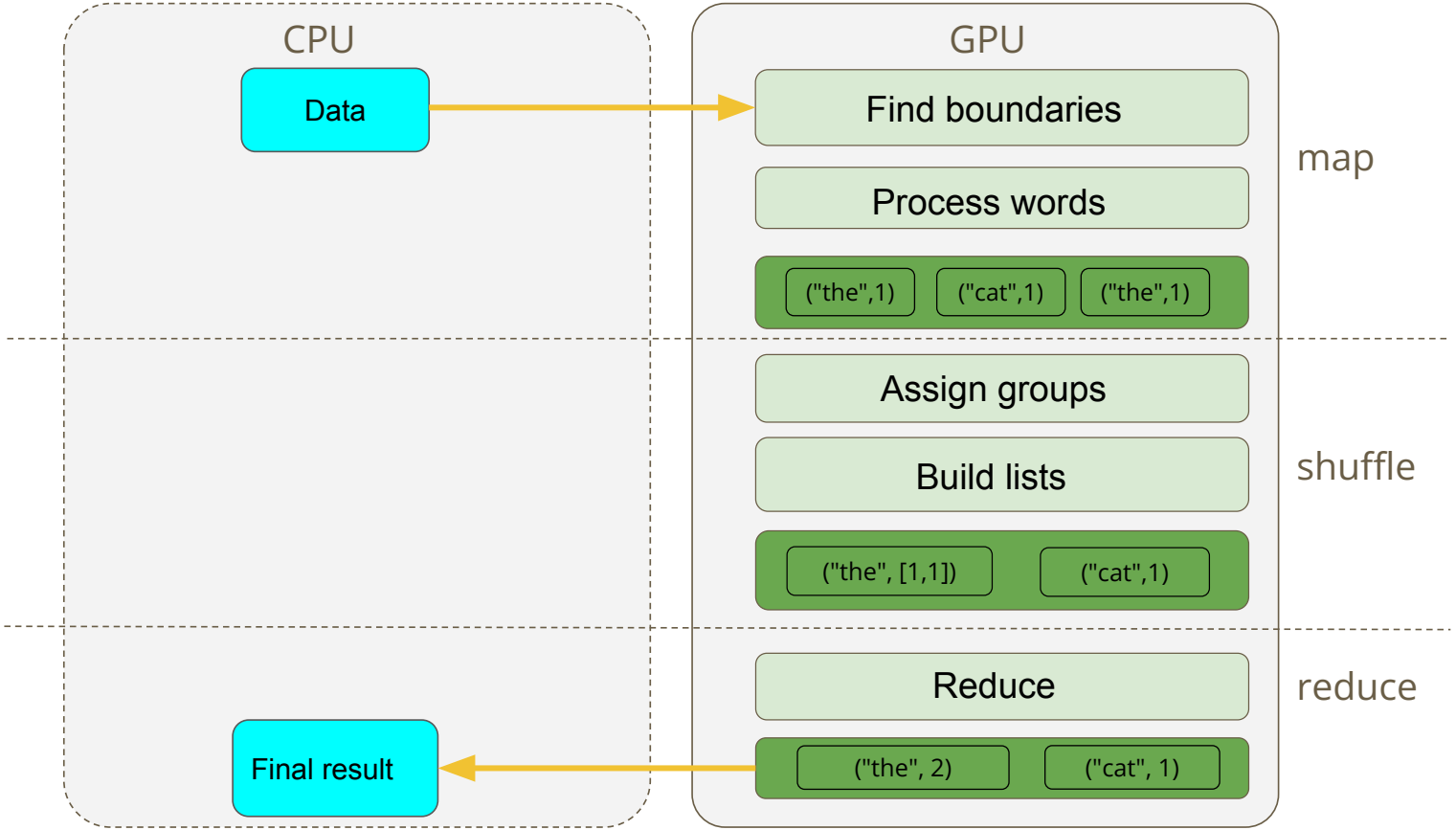
Proposed solution - [Shuffle]



Proposed solution - [Reduce]



Proposed solution- Cuda



Challenges encountered

- Pthread & OpenMP : High Contention on Global Data Structures
  - Implemented Thread-Local Storage, Assigned a separate vector (local buffer) for each thread to store intermediate Map results.
- MPI : I/O Bottleneck with Single-Rank File Reading
  - Broadcasting data from Rank 0 is slow, implemented Parallel I/O where each rank seekg to its offset and reads concurrently
- CUDA : Standard GPU reduction often relies on thrust::sort O(Nlog N) to group keys because std::map is unavailable on devices.
  - Implemented a custom Hash Table with MurmurHash3 and Linear Probing.Used `atomicCAS` to construct (Key, List<Values>) groups in parallel, achieving O(N) average complexity and bypassing the sorting phase.

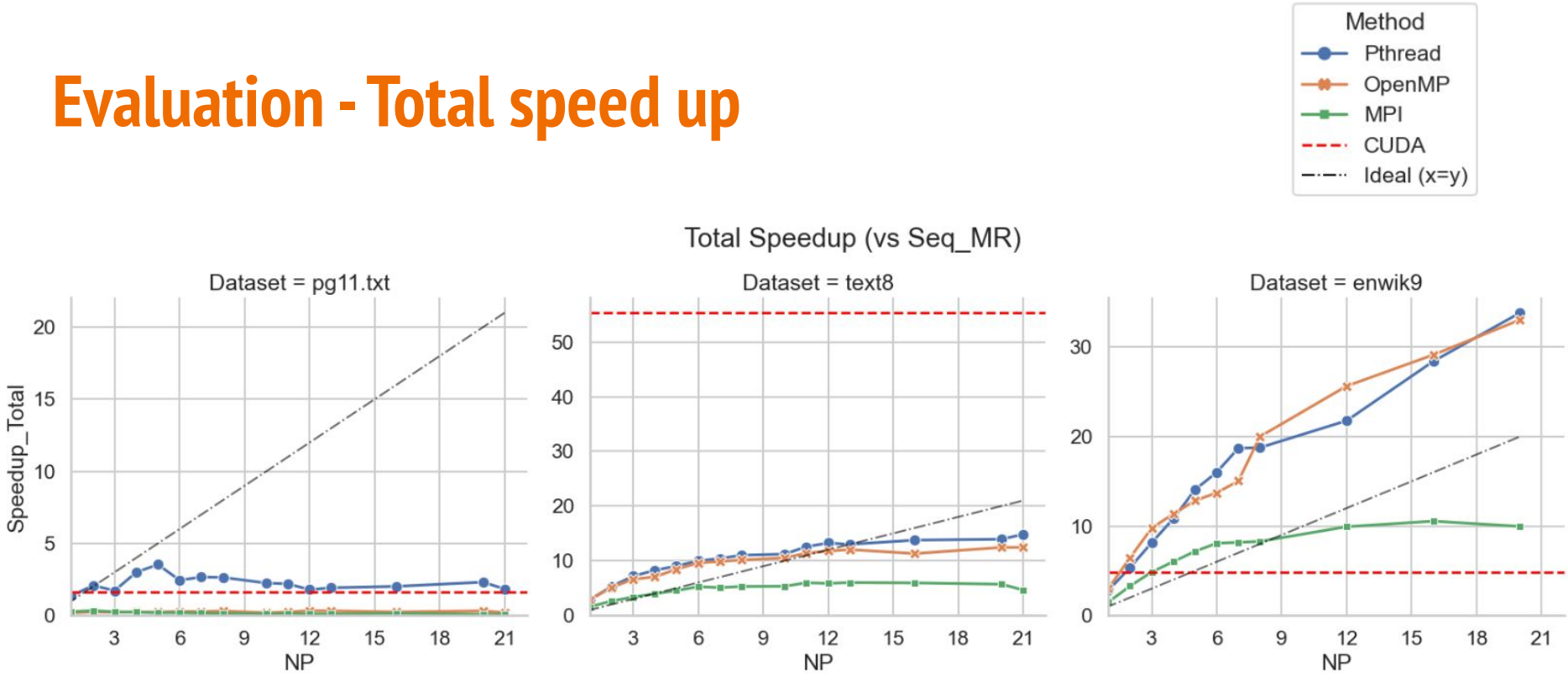
Platform

- Environment : Windows Subsystem for Linux 2 (WSL 2)
- OS : Ubuntu 24.04.3 LTS
- Kernel : 6.6.87.2-microsoft-standard-WSL2
- CPU : 13th Gen Intel(R) Core(TM) i5-13500HX core 14 process 20
- GPU : NVIDIA GeForce RTX 4060 (CUDA 13.0)

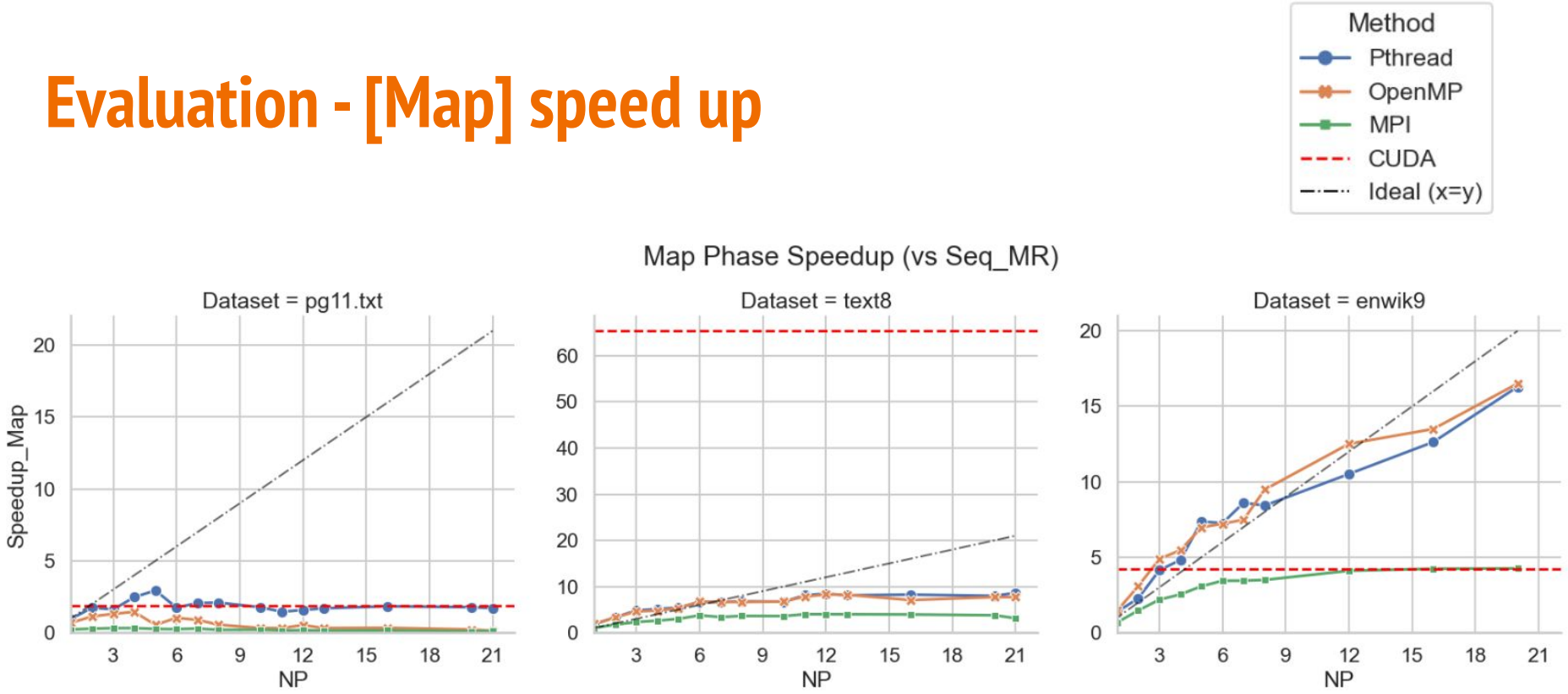
Dataset

- Dataset
  - pg11.txt : Small (170 KB)
  - text8.txt : Medium (100 MB)
  - enwik9.txt : Large(1GB)

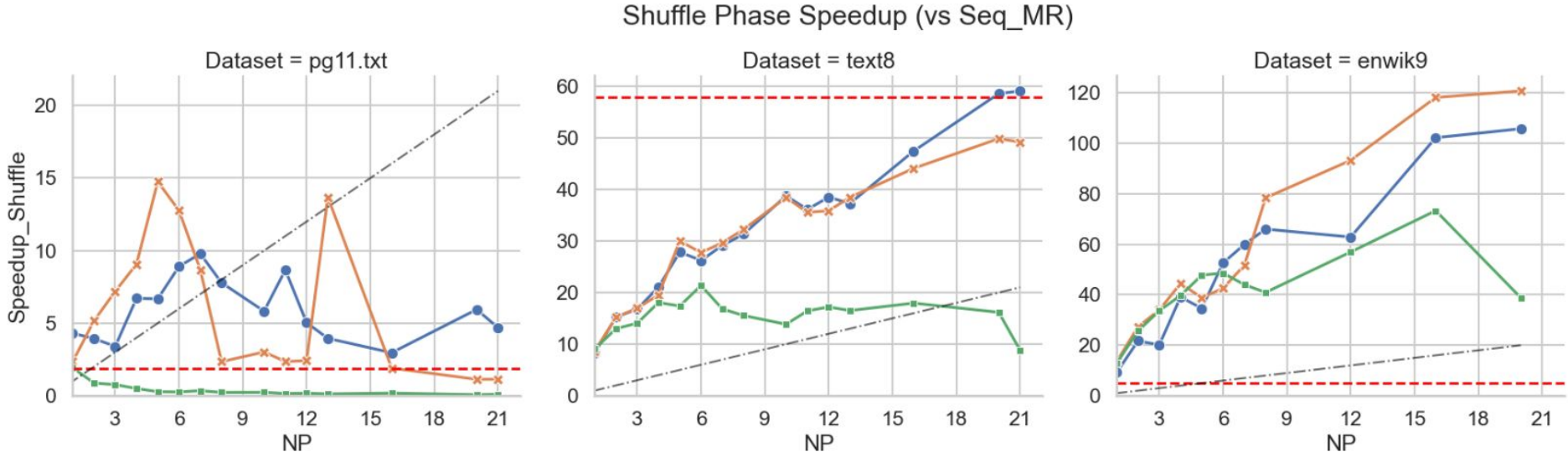
Evaluation - Total speed up



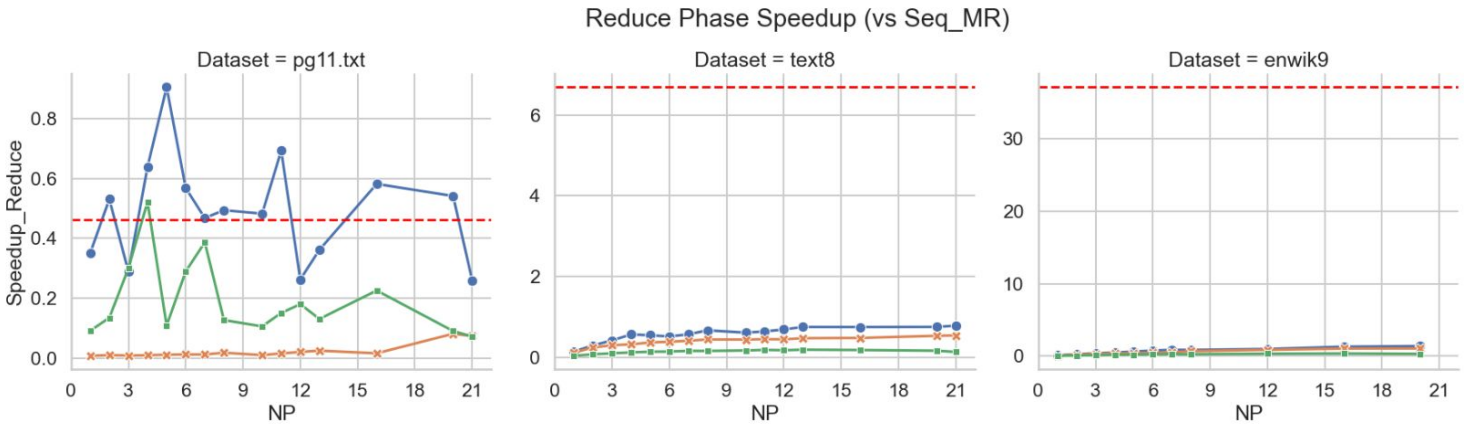
Evaluation - [Map] speed up

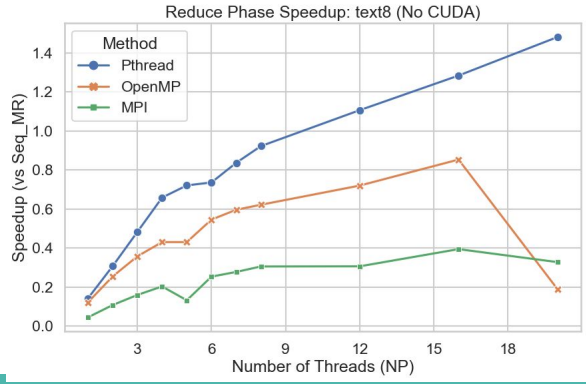


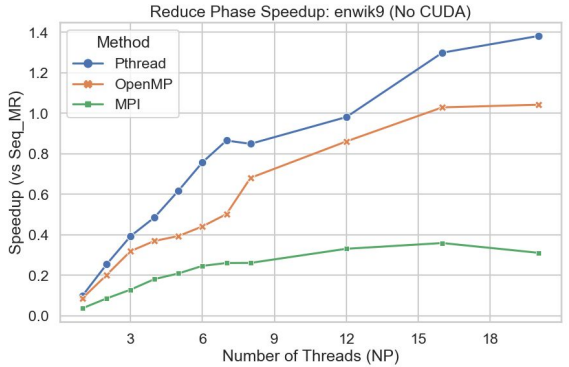
Evaluation - [Shuffle] speed up



Evaluation - [Reduce] speed up







Conclusion

- **Performance vs. Dataset Size**
  - **Parallelization Overhead:** For small datasets (e.g., pg11.txt, 170KB), parallel overhead (thread creation, context switching, memory transfer) outweighs computational benefits.
- Key Optimization Strategies
  - **CPU (Pthread/OpenMP): Eliminated lock contention** on global data structures by implementing **Thread-Local Storage (TLS)**, merging results only in the Reduce phase.
  - **Distributed (MPI):** Overcame the single-rank I/O bottleneck by implementing **Parallel I/O**, allowing all ranks to calculate offsets and read files concurrently.
  - **GPU (CUDA):** Replaced the standard sorting-based reduction (O(N (log N)) with a custom **Atomic Hash Table** (O(N)), significantly improving throughput on massive datasets

Contributions of each member

Team Member	Assigned Responsibilities
陳姿蓉	簡報製作, OpenMP,Pthread
林孝欣	簡報製作, CUDA
朱冠熙	簡報製作, MPI

Q & A

Reference

Dataset:

Small (170 KB) :

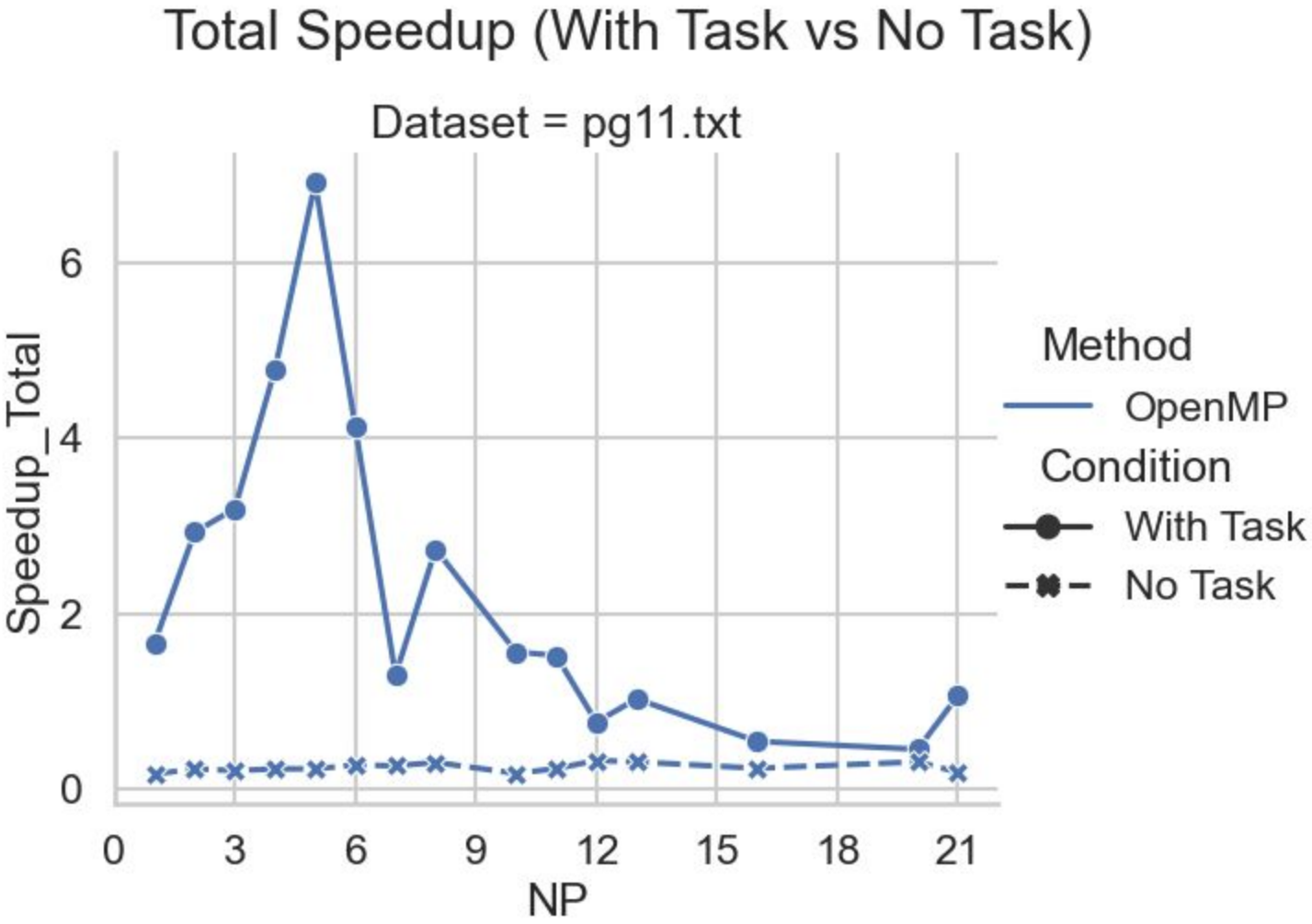
https://www.gutenberg.org/ebooks/11

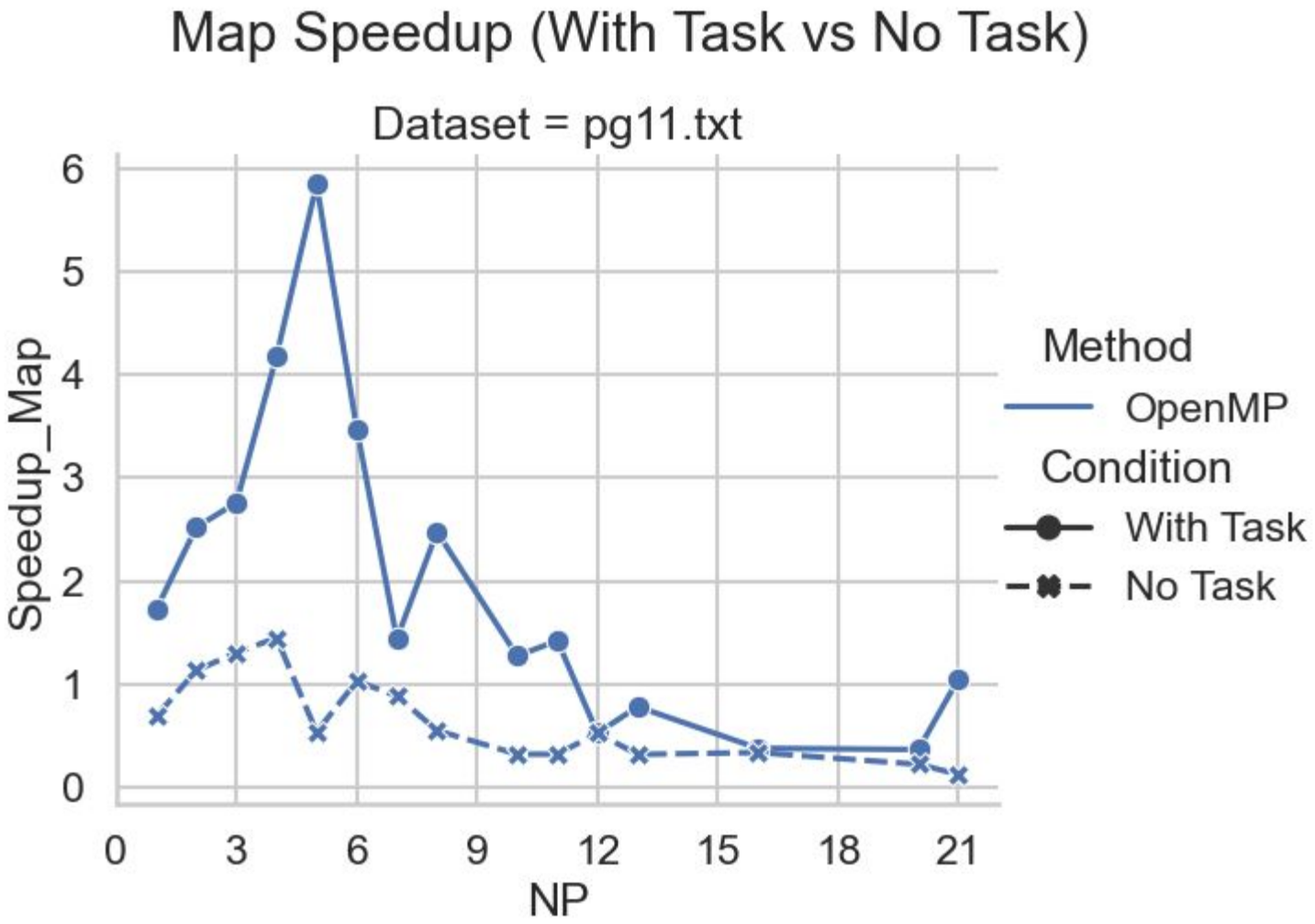
Medium (100 MB) :

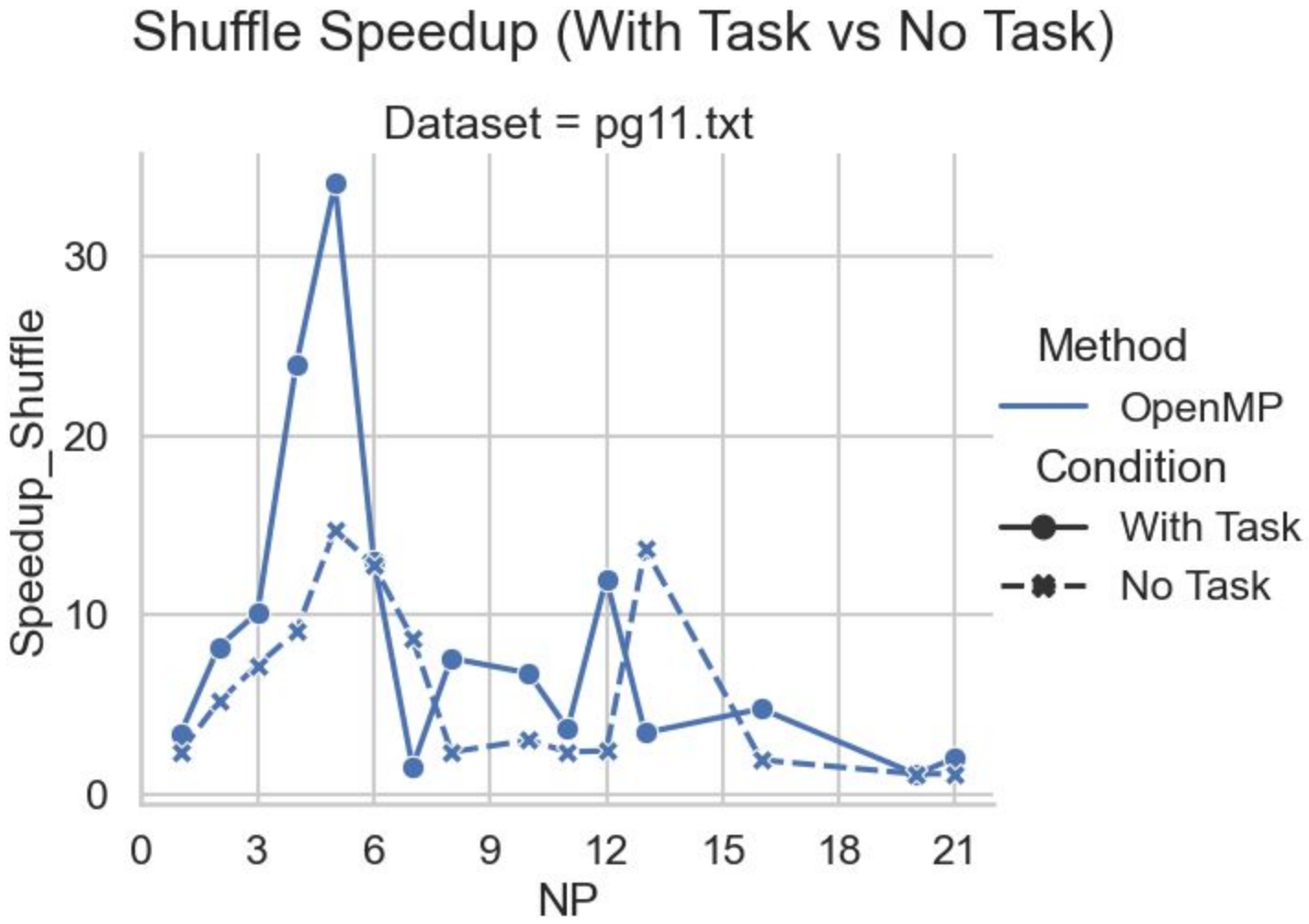
https://www.kaggle.com/datasets/yorkyong/text8-zip?resource=download

Large(1GB):

https://mattmahoney.net/dc/textdata.html


Total Speedup (With Task vs No Task)
Dataset = pg11.txt
Speedup_Total
0
2
4
6
0
3
6
9
12
15
18
21
NP
Method
OpenMP
Condition
With Task
No Task


Map Speedup (With Task vs No Task)
Dataset = pg11.txt
Speedup_Map
NP
0
1
2
3
4
5
6
0
3
6
9
12
15
18
21
Method
OpenMP
Condition
With Task
No Task


Shuffle Speedup (With Task vs No Task)
Dataset = pg11.txt
Speedup_Shuffle
0
10
20
30
0
3
6
9
12
15
18
21
NP
Method
OpenMP
Condition
With Task
No Task

Reduce Speedup (With Task vs No Task)

